SIMATS ENGINEERING
CO5-ASSESSMENT TOOL1-Design Task

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**Venu G**
Name	**192424390**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 40%

Time Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I _____Venu G(192424390)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered
to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

RUBRICS FOR CALCULATION

Evaluation Criteria	**Problem Understanding & Requirement Analysis**	**System Architecture Design**	**Use of PySpark & RDD Operations**	**Intelligent Agent Integration**	**Scalability & Distributed Computing Design**	**Data Flow & Processing Pipeline**	**Innovation & Creativity**	**Clarity, Presentation & Documentation**
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	**Excellent (5)**	**Good (4)**	**Average (3)**	**Poor (2)**
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/ actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Distributed AI Systems with PySpark and RDDs

Five End-to-End System Design Studies: Architecture, Data Flow, and Multi-Agent Integration

1. Real-Time Social Media Analytics & Recommendation System

1.1 Problem Overview

The goal is to ingest high-velocity social media streams (posts, likes, shares, comments) and continuously generate personalized recommendations — content, connections, or trending topics — while the system scales horizontally and tolerates node failures. PySpark's RDD abstraction provides the fault-tolerant, in-memory distributed processing layer; an agent layer sits above it to make adaptive, goal-directed decisions (what to recommend, to whom, and when).

1.2 System Architecture

The architecture is organized into five layers:

Layer	Responsibility	Key Technology
Ingestion	Capture posts/events from social APIs and webhooks	Kafka / Kinesis producers
Stream Processing	Micro-batch ingestion and windowed aggregation	Spark Structured Streaming → RDD/DStream
Distributed Compute	Feature extraction, joins, filtering at scale	PySpark RDD transformations
Agent & Intelligence	Recommendation policy, ranking, personalization	Multi-agent layer (Recommender, Trend, Moderation agents)
Serving	Low-latency delivery of recommendations	Redis / REST API / push notifications

1.3 Data Flow

- Raw events land in a Kafka topic partitioned by user-region for locality.
- Spark Structured Streaming consumes micro-batches; each micro-batch is converted to an RDD for custom transformations not easily expressed declaratively (e.g., custom similarity scoring).

- RDD transformations perform: text cleaning, tokenization, hashtag/entity extraction, and engagement-weighted scoring.
- Aggregated features are joined with a cached user-profile RDD (broadcast for small profiles, partitioned join for large ones).
- The Agent layer consumes the enriched RDD output to compute per-user recommendation candidates and writes them to a low-latency store.
- A feedback loop streams click/engagement signals back into the pipeline to retrain ranking models incrementally.

1.4 Core RDD Processing Example

```
# Ingest micro-batch as RDD and compute engagement-weighted topic scores
def process_batch(rdd, user_profiles_bc):
    if rdd.isEmpty():
        return
    parsed = rdd.map(parse_json_event) \
              .filter(lambda e: e['type'] in ('post', 'like', 'share'))
    scored = parsed.map(lambda e: (e['user_id'], (e['topic'], engagement_weight(e)))) \
                .reduceByKey(lambda a, b: merge_topic_scores(a, b))

    enriched = scored.map(lambda kv: (kv[0], kv[1], user_profiles_bc.value.get(kv[0])))

    recommendations = enriched.mapPartitions(agent_rank_partition)

    recommendations.foreachPartition(write_to_redis)
```

1.5 Agent Integration

- Recommender Agent: consumes scored RDD partitions via mapPartitions, applies a lightweight ranking model per partition to avoid driver bottlenecks.
- Trend Detection Agent: runs windowed RDD aggregations (5-min / 1-hr) to detect spikes using a z-score over rolling counts, publishing 'trending' signals other agents subscribe to.
- Moderation Agent: filters candidate recommendations against a broadcast blocklist RDD before the serving layer.
- Agents communicate through a shared event bus (Kafka topics) rather than direct RPC, keeping them decoupled from Spark's driver lifecycle.

1.6 Scalability & Fault Tolerance

- RDD lineage allows automatic recomputation of lost partitions without checkpoint replay for most stages.
- Checkpointing is enabled on stateful streaming aggregations (rolling counts) to bound lineage growth.
- Dynamic executor allocation scales compute with ingestion rate; Kafka partition count bounds maximum parallelism.

- User-profile broadcast variables are refreshed periodically to avoid stale personalization without re-shuffling the whole dataset.

2. Distributed Intelligent Agent System for Large-Scale Optimization

2.1 Problem Overview

Many optimization problems (route planning, resource scheduling, network flow) are too large for a single machine to search exhaustively. This design uses PySpark to distribute the search space across a cluster while a set of collaborating agents — Explorer, Evaluator, and Coordinator — iteratively refine candidate solutions.

2.2 Agent Roles

Agent	Role	Spark Mechanism
Explorer Agents	Generate candidate solutions (e.g., mutate/crossover in a genetic algorithm, or explore neighborhoods in local search)	RDD.flatMap over partitioned candidate population
Evaluator Agents	Score candidates against the objective/constraint functions	RDD.map + broadcast of problem constraints
Coordinator Agent	Aggregates results, selects survivors, decides convergence, redistributes work	RDD.reduce / takeOrdered on driver, re-partition for next round

2.3 Architecture

The system runs as an iterative Spark job: each iteration is a generation/round. The population of candidate solutions is stored as an RDD partitioned across executors. Explorer and Evaluator agent logic is embedded as functions executed inside RDD transformations (mapPartitions is preferred over map to allow agents to maintain local state and batch their evaluation calls, e.g., to a scoring service or local heuristic).

- Driver process hosts the Coordinator Agent, orchestrating rounds and applying global stopping criteria.
- Executors host stateless Explorer/Evaluator logic; any required shared knowledge (best-known solution, constraints) is distributed via broadcast variables.
- An accumulator tracks the global best score across partitions to allow early-stopping checks without a full collect().

2.4 Collaboration Protocol

- Round start: Coordinator broadcasts current best solution and search parameters (mutation rate, temperature, etc.).
- Explore: Explorer agents run in parallel across partitions, each producing a local batch of candidates via mapPartitions.
- Evaluate: Evaluator agents score candidates; results are combined with reduceByKey/aggregateByKey when candidates are grouped by sub-problem.
- Select: Coordinator uses takeOrdered(k) to pick top-k candidates without a full shuffle-and-sort of the entire population.
- Converge check: Coordinator compares improvement over the accumulator's tracked best score against a tolerance threshold; if below tolerance for N rounds, the job terminates.

2.5 Distributed Optimization Code Example

```
def run_generation(population_rdd, constraints_bc, best_bc):
    def explore_and_evaluate(partition):
        local_best = best_bc.value
        constraints = constraints_bc.value
        for candidate in partition:
            child = explorer_agent.mutate(candidate, local_best)
            score = evaluator_agent.score(child, constraints)
            yield (child, score)

    scored = population_rdd.mapPartitions(explore_and_evaluate)
    survivors = scored.takeOrdered(POPULATION_SIZE, key=lambda x: -x[1])
    return survivors

# Driver-side coordinator loop
best = None
for gen in range(MAX_GENERATIONS):
    survivors = run_generation(population_rdd, constraints_bc, best_bc)
    best_candidate = max(survivors, key=lambda x: x[1])
    if converged(best, best_candidate):
        break
    best = best_candidate
    best_bc = sc.broadcast(best)
    population_rdd = sc.parallelize([c for c, _ in survivors], NUM_PARTITIONS)
```

2.6 Fault Tolerance & Scalability

- RDD lineage recomputes any lost partition of the candidate population deterministically, since agent transformations are pure functions of (candidate, broadcast state).
- Partition count scales with cluster size and problem decomposability; sub-problems that don't decompose well cap parallel speedup (Amdahl's law consideration noted explicitly in design review).
- Broadcast variables avoid re-shipping constraints/best-known solution on every task, reducing network overhead as the population grows.

3. Big Data AI Pipeline: Preprocessing, Feature Engineering & Training

3.1 Pipeline Stages

A complete distributed AI pipeline is decomposed into four RDD-driven stages, each independently scalable and checkpointable:

Stage	RDD Operations Used	Output
Ingestion & Cleaning	map, filter, distinct	Cleaned raw-record RDD
Feature Engineering	flatMap, reduceByKey, aggregateByKey, join	Feature-vector RDD
Partition & Sample	repartition, sampleByKey	Balanced train/validation RDDs
Model Training	mapPartitions (local model fit), treeReduce (gradient aggregation)	Trained model artifact

3.2 Preprocessing & Feature Engineering

- Raw records are deduplicated with distinct() and malformed records filtered out via filter() with schema validation.
- Categorical features are hashed with flatMap to explode multi-valued fields, then aggregated with reduceByKey to build sparse feature counts.
- Numerical features are joined against a reference RDD (e.g., historical statistics) using join() or, when one side is small, a broadcast join to avoid an expensive shuffle.
- aggregateByKey is used over groupByKey wherever a combine function exists, since it performs map-side pre-aggregation and greatly reduces shuffle volume.

3.3 Distributed Training Approach

For models that support data-parallel training (e.g., linear models via gradient descent, or ensembles), each partition computes a local gradient or fits a local model on its shard; results are combined with treeReduce, which aggregates in a tree pattern to avoid overloading the driver as a single reduction point.

```
features_rdd = raw_rdd.map(clean_record) \
                .filter(is_valid) \
                .flatMap(explode_categorical) \
                .map(lambda r: (r['id'], to_feature_vector(r))) \
                .aggregateByKey(zero_vec, seq_op=merge_vec, comb_op=merge_vec)
```

```
train_rdd, val_rdd = features_rdd.randomSplit([0.8, 0.2], seed=42)
train_rdd = train_rdd.repartition(NUM_PARTITIONS).cache()

def local_gradient(partition):
    X, y = build_matrix(list(partition))
    yield compute_gradient(X, y, weights_bc.value)

for epoch in range(EPOCHS):
    grad = train_rdd.mapPartitions(local_gradient).treeReduce(lambda a, b: a + b)
    weights = update_weights(weights_bc.value, grad, lr=LEARNING_RATE)
    weights_bc = sc.broadcast(weights)
```

3.4 Scalability & Fault Tolerance

- cache()/persist() on the training RDD avoids recomputing the feature pipeline every epoch, at the cost of memory; persist(DISK_ONLY) is used as a fallback under memory pressure.
- Checkpointing (sc.setCheckpointDir) truncates long RDD lineages built up over many training epochs, preventing stack-overflow-style recomputation chains on failure.
- treeReduce's tree-shaped aggregation bounds driver load as cluster size grows, unlike a flat reduce().
- Skewed keys in feature aggregation are mitigated with salting (splitting a hot key into sub-keys before reduceByKey, then merging) to keep partitions balanced.

4. Real-Time Decision-Making System (Smart Traffic / Healthcare)

4.1 Overview

This design targets applications where decisions must be made within seconds of new sensor data arriving — for example, adjusting traffic signal timing from live camera/inductive-loop feeds, or flagging a deteriorating patient from streaming vitals. It combines RDD-based micro-batch processing with a decision agent that must act under strict latency budgets.

4.2 Architecture & Data Flow

- Edge devices (traffic sensors / bedside monitors) publish readings to a message broker (Kafka/MQTT bridge) with a device-id partition key.
- Spark Structured Streaming consumes micro-batches at a short trigger interval (e.g., 2–5 seconds) and exposes each batch as an RDD for custom logic.
- A Decision Agent evaluates each RDD partition against rule-based thresholds and a lightweight learned model (e.g., anomaly score), combining both for robustness.
- Decisions (e.g., 'extend green phase on Elm & 5th', or 'escalate patient 4021 to nurse review') are pushed to an actuation topic consumed by downstream control systems or alerting services.
- A slower-path RDD job re-aggregates the same data over longer windows (hourly/daily) to retrain thresholds and detect concept drift, keeping the real-time path lightweight.

4.3 Example: Smart Traffic Signal Control

```
def process_traffic_batch(rdd):
    if rdd.isEmpty():
        return
    readings = rdd.map(parse_sensor_reading)

    by_intersection = readings.map(lambda r: (r['intersection_id'], r)) \
                              .groupByKey()

    decisions = by_intersection.mapValues(traffic_agent.decide) \
                               .filter(lambda kv: kv[1] is not None)

    decisions.foreachPartition(publish_signal_commands)

def decide(readings):
    congestion = compute_congestion_score(readings)
    if congestion > THRESHOLD:
        return {'action': 'extend_green', 'seconds': scale_extension(congestion)}
    return None
```

4.4 Healthcare Variant Notes

- Patient vitals streams are partitioned by patient-id to preserve per-patient temporal ordering within a partition.

- Decision Agent applies clinically-reviewed rule thresholds first (hard safety bounds) and only then consults a learned risk model — rules are never overridden by the model, only supplemented, for auditability and safety.
- All triggered alerts are written to an immutable audit-log RDD sink in addition to the real-time actuation path, satisfying traceability requirements.

4.5 Latency, Scalability & Fault Tolerance

- Short micro-batch intervals trade throughput efficiency for latency; partition count is tuned so each batch's RDD stages complete well within the trigger interval.
- Backpressure settings cap ingestion rate per batch to prevent runaway batch sizes during traffic/sensor bursts.
- Write-ahead logs (Structured Streaming checkpointing) allow exactly-once recovery of in-flight micro-batches after executor failure, critical for healthcare alerting correctness.
- Stateless decision logic (pure functions of the current window) keeps recovery fast, since no long-lived per-partition state needs to be replayed.

5. Cloud-Based Distributed AI Application

5.1 Overview

This design targets a cloud-native deployment (Kubernetes-on-EKS/GKE/AKS or a managed Spark service such as Databricks/EMR) that supports large-scale analytics, elastic resource allocation, and an adaptive learning loop that improves models as more data arrives, without manual retraining pipelines.

5.2 Cloud Architecture

Tier	Component	Cloud Service Pattern
Storage	Raw + curated data lake	Object storage (S3/GCS/ADLS) with partitioned Parquet
Compute	Elastic Spark cluster	Kubernetes Spark operator / managed Spark autoscaling
Orchestration	Pipeline & retraining scheduling	Workflow orchestrator (Airflow) triggering Spark jobs
Agent Layer	Adaptive learning + serving agents	Containerized microservices reading model registry
Model Registry	Versioned model artifacts	Managed registry (e.g., MLflow) backed by object storage

5.3 Dynamic Resource Allocation

- Spark's dynamic allocation (spark.dynamicAllocation.enabled) is combined with the cluster autoscaler so executor pods scale with pending task backlog rather than a fixed cluster size.
- RDD partition counts are set relative to available cores (typically 2–4x total cores) so that autoscaling has enough parallel units to schedule as new nodes join.
- Spot/preemptible instances are used for stateless, easily-recomputable RDD stages; on-demand instances are reserved for stages holding checkpointed or stateful data, to bound the cost of preemption-triggered recomputation.

5.4 Adaptive Learning Loop

- Incoming data lands in the data lake and is periodically processed into feature RDDs, as in Section 3's pipeline.
- A monitoring agent tracks prediction-vs-outcome drift using a rolling RDD aggregation; when drift exceeds a threshold, it triggers an automated retraining job via the orchestrator.

- Newly trained models are registered with a version tag; a champion/challenger rollout compares the new model's held-out performance to the current production model before promoting it.
- Serving agents pull the current champion model reference from the registry, allowing zero-downtime model updates.

5.5 Example: Dynamic Allocation Configuration

```
spark = SparkSession.builder \
    .appName('cloud-adaptive-ai') \
    .config('spark.dynamicAllocation.enabled', 'true') \
    .config('spark.dynamicAllocation.minExecutors', '4') \
    .config('spark.dynamicAllocation.maxExecutors', '200') \
    .config('spark.dynamicAllocation.shuffleTracking.enabled', 'true') \
    .config('spark.sql.shuffle.partitions', '400') \
    .getOrCreate()

sc = spark.sparkContext
sc.setCheckpointDir('s3://ml-platform/checkpoints/')

# Drift-triggered retraining check (simplified)
drift_score = predictions_rdd.map(lambda r: (r['model_version'], compute_error(r))) \
                        .aggregateByKey((0.0, 0), seq_add_err, comb_add_err) \
                        .mapValues(lambda x: x[0] / x[1]) \
                        .collectAsMap()

if drift_score.get(CURRENT_MODEL, 0) > DRIFT_THRESHOLD:
        trigger_retraining_dag()
```

5.6 Scalability & Resilience

- Stateless microservice agents behind a load balancer scale independently of the Spark cluster, isolating serving-path availability from batch job load.
- Multi-zone deployment plus checkpointing to durable object storage ensures both compute (Spark) and state (checkpoints, model registry) can recover after a zone failure.
- Cost and performance are monitored per pipeline stage so autoscaling policies and spot-instance usage can be tuned per workload rather than cluster-wide.